Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.

Qingyang Zhang, Haitao Wu, Changqing Zhang, Peilin Zhao, and Yatao Bian. Right question is already half the answer: Fully unsupervised LLM reasoning incentivization. *arXiv preprint arXiv:2504.05812*, 2025.

Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The surprising effectiveness of negative reinforcement in LLM reasoning. *arXiv preprint arXiv:2506.01347*, 2025.

A APPENDIX

A.1 CONTRIBUTIONS

We summarize individual contributions as follows:

- Xumeng Wen conducted all rollout and verification experiments and led the $Pass@K$ analysis on mathematical benchmarks.
- Zihan Liu performed meticulous examination of numerous mathematical reasoning cases and verified the reliability of the LLM-as-a-CoT-Judge paradigm.
- Shun Zheng designed this research, conducted RLVR experiments, identified CoT correctness as the key factor, built the theoretical framework, and wrote the manuscript.
- Shengyu Ye developed the EvalHub tool (Ye, 2025), supporting most rollout and verification experiments, and performed Pass@K experiments for AceReason-Nemotron-7B.
- Zhirong Wu conducted independent rollout studies and provided insightful thoughts during group discussions.
- Yang Wang provided crucial infrastructure support, enabling all experiments through AMD MI300 GPUs.
- Zhijian Xu contributed to training analysis and proposed the visualization of $P(\text{CC}|\text{CA})$.
- Xiao Liang shared valuable insights regarding mathematical benchmarks, training question selection, and effective mathematical verifiers.
- Junjie Li participated actively in group discussions and shared RL experiences.
- Ziming Miao provided additional infrastructure support.
- Jiang Bian and Mao Yang supervised the project and provided comprehensive support to the entire team.

A.2 DATA SOURCES

For math benchmarks studied in this paper, we leverage the following data sources: AIME 2025[1], AIME 2024[2], Math-500[3], AMC23[4], Minerva[5].

For training and evaluation of DAPO (Yu et al., 2025), we reuse their training data (`https://huggingface.co/datasets/BytedTsinghua-SIA/DAPO-Math-17k`) and processed version of AIME 2024 (`https://huggingface.co/datasets/BytedTsinghua-SIA/AIME-2024`). Please note that they have duplicated questions multiple times and explained in the dataset page that the purpose is to be compatible with an old version of VERL (Sheng et al., 2025). We reuse the prompt template of DAPO to evaluate their $Pass@K$ and $CoT\text{-}Pass@K$ performance on other benchmarks.

[1] `https://huggingface.co/datasets/opencompass/AIME2025`
[2] `https://huggingface.co/datasets/HuggingFaceH4/aime_2024`
[3] `https://huggingface.co/datasets/HuggingFaceH4/MATH-500`
[4] `https://huggingface.co/datasets/math-ai/amc23`
[5] `https://huggingface.co/datasets/math-ai/minervamath`

Besides, we follow the official LiveCodeBench repository[6] to perform Pass@K evaluations on competitive coding.

A.3 LLM-AS-A-COT-JUDGE FOR MATH REASONING

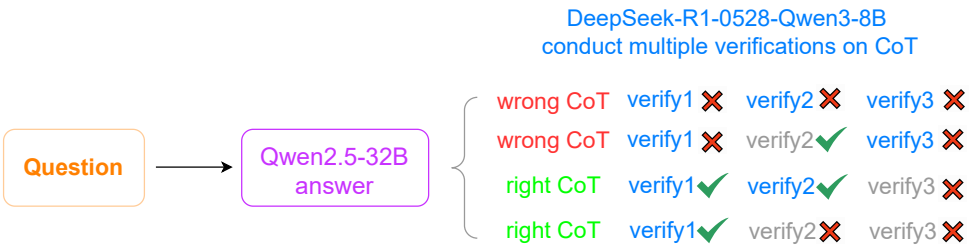

Figure 7: An intuitive diagram to illustrate the benefits of our multi-verification system: simultaneously considering *any-correct*, *all-correct*, and *majority-correct* helps us to mitigate false positives and false negatives within individual verifications.

We use a much more specialized LLM on mathematical reasoning (DeepSeek-R1-0528-Qwen3-8B) as the verifier to examine the reasoning steps of base LLM, Qwen2.5-32B. Meanwhile, we also acknowledge the existence of verification errors and manually checked many of them to confirm the reliability of this verification. To further mitigate potential verification errors, we design a multi-verification approach, as shown in Figure 7). For each reasoning CoT, we conduct multiple verifications independently and calculate three aggregation metrics:

- *All-correct*: Chains of Thought that pass all verification attempts
- *Majority-correct*: Chains of Thought that pass most verification attempts
- *Any-correct*: Chains of Thought that pass at least one verification attempt (capturing potential error recovery cases)

This multi-verification approach can ensure us to have a comprehensive view of CoT correctness because the *all-correct* strategy mitigates false positives and the *any-correct* option reduces false negatives. Let p_{fp} and p_{fn} represent the per-attempt false positive and false negative rates, respectively. For n independent verification attempts, we observe:

- *All-correct*: The false positive rate decays exponentially as p_{fp}^n
- *Any-correct*: The false negative rate decays exponentially as p_{fn}^n

In our study, we employ $n = 3$ verification attempts for each CoT.

Moreover, we provide the prompt template used for DeepSeek-R1-0528-Qwen3-8B as follows.

[6] `https://github.com/LiveCodeBench/LiveCodeBench`

Our Prompt Template for Verifier DeepSeek-R1-0528-Qwen3-8B

You are an expert in mathematics and logical reasoning. Your task is to evaluate the correctness of a solution to a given math problem, with a **strong emphasis on the reasoning process**, not just the final answer.
Below is the **Problem** and the **Solution (Provided by another AI model)**:
—
Problem:
{{question}}
Solution (Provided by another AI model):
{{solution}}
—
Please perform the following tasks:
1. **Analyze the solution step-by-step**, paying close attention to: - Computational accuracy - Logical consistency - Conceptual understanding - Whether the reasoning is valid and complete
2. **Identify any issues or errors in the reasoning**, even if the final answer is correct. Classify them into the following categories (if applicable): - **Calculation Error**: Mistakes in arithmetic, algebraic manipulation, or numerical computation. - **Logical Error**: Invalid reasoning, flawed logic, or incorrect inference. - **Conceptual Error**: Misunderstanding or misuse of mathematical concepts or definitions. - **Omission / Incompleteness**: Missing steps, incomplete justification, or not addressing all parts of the question. - **Other**: Any other type of error that does not fit into the above categories.
3. **Provide a final judgment** on whether the solution is logically sound and free of errors in reasoning.
Please format your response as follows:
—
Issues Identified:
- [Issue 1]: [Classification] - [Brief explanation] - [Issue 2]: [Classification] - [Brief explanation] - ...
Let's think step by step and output your final judgment within \boxed{}
\boxed{yes} or \boxed{no}

A.4 Revisiting Pass@K Experiments for Skywork-OR1

Skywork-OR1 (He et al., 2025) has generously shared their complete training recipes, claiming to enhance distilled LLMs with more powerful reasoning capabilities through RLVR. Therefore, we conduct the Pass@K experiments on their models to understand how RLVR could improve distilled LLMs.

Figure 8 shows the Pass@K comparisons between Skywork-OR1-7B and DeepSeek-R1-Distill-Qwen-7B on LiveCodeBench-v6. We can also observe a significant improvement of both sampling efficiency (Pass@1) and reasoning boundary (Pass@K, K up to 1024). This conclusion is consistent with the observations for Figure 3 in the main paper.

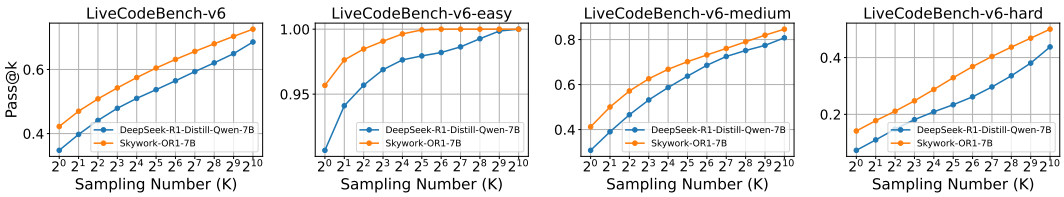

Figure 8: Comparisons of Pass@K on LiveCodeBench-v6 and its different difficulty-level subsets. Here the distilled LLM is DeepSeek-R1-Distill-Qwen-7B, and the post-RLVR model is Skywork-OR1-7B.

However, in math domains, applying RLVR to distilled LLMs seems to merely deliver sampling efficiency improvements. As shown in Figure 9, we observe that even using the CoT-Pass@K metric,